UNIVERSITY OF ZAGREB
Faculty of Electrical Engineering and Computing

THESIS no. xxxx

Control system for thermal processes in data-center mock-up

Ema Rozmajer

Zagreb, May 2026.

UNIVERSITY OF ZAGREB
Faculty of Electrical Engineering and Computing

THESIS no. xxxx

Control system for thermal processes in data-center mock-up

Ema Rozmajer

Zagreb, May 2026.

Acknowledgements

I would like to express my sincere gratitude to Prof. Stjepan Bogdan, Assist. Prof. Gordan Vasiljević, and Assoc. Prof. Tamara Petrović for their guidance, support, and valuable advice throughout the development of this thesis. Their knowledge, expertise, and continuous encouragement greatly contributed to the completion of this work.

Table of Contents

Introduction *1*

1. Mathematical Modelling of the Cooling System *2*

1.1. Pressure Equations **3**

1.2. Temperature Equations **5**

1.3. Simulink Model **7**

1.4. Simulation Results **8**

2. Mock-Up of the Cooling System *15*

2.1. System Components **16**

2.1.1. Controller and Motor Shield 16

2.1.2. Sensors 17

2.1.3. Actuators 18

2.1.4. Additional Electronic Components 19

2.2. System Configuration and Wiring Diagram **19**

2.3. Control System Design **20**

2.3.1. Speed Control 20

2.3.2. Pressure Control 21

2.3.3. Temperature Control 22

2.3.4. PID Controller 22

2.4. Software Implementation **23**

3. Results and Comparison *24*

3.1. Speed Control **24**

3.2. Pressure Response Analysis **26**

3.3. Temperature Control Analysis **28**

3.4. Integrated System Performance **30**

Conclusion *32*

References *33*

Abstract *34*

Summary *35*

Abbreviations *36*

Appendix A – Arduino Implementation *37*

Appendix B – MATLAB Implementation *48*

Introduction

The increasing demand for data processing and storage has led to the rapid development of data centers, where efficient thermal management is essential for reliable system operation. Electronic equipment within data centers generates significant amounts of heat, which must be effectively removed to maintain safe operating conditions and ensure optimal performance.

Cooling systems play a crucial role in keeping temperature stability and preventing overheating. Proper airflow management, including the separation of hot and cold air regions, is commonly used in data centers to improve cooling efficiency.

In this work, a control system for thermal processes in a data center environment is investigated using a simplified mock-up model. The aim is to develop and analyze a control strategy capable of regulating temperature and pressure by adjusting airflow within an enclosed space.

The system is modeled using fundamental principles of thermodynamics, and the mathematical model is implemented and simulated in the MATLAB/Simulink environment. In addition, a physical mock-up model is constructed to validate the behavior of the system under real conditions.

The control system is based on feedback loops that regulate fan speed to maintain desired temperature and pressure conditions. The results obtained from simulation and experimental testing provide insight into the system dynamics and demonstrate the effectiveness of the proposed control approach.

The work is structured as follows. Chapter 1 presents the mathematical modelling of the cooling system, including the pressure and temperature equations, the Simulink implementation, and the simulation results. Chapter 2 describes the physical mock-up model, including the system components, wiring configuration, control system design, and software implementation. Chapter 3 presents the experimental results and comparison between the mathematical model and the mock-up model, including speed, pressure, and temperature control analysis. Finally, the conclusion summarizes the obtained results and the overall performance of the implemented control system.

1. Mathematical Modelling of the Cooling System

Mathematical modelling represents a crucial step in the analysis and design of control systems. The main purpose of modelling is to describe the behavior of a physical system using mathematical equations, which enables better understanding, analysis, and control of the process [1].

In this work, a mathematical model of a closed-space cooling system is developed. The schematic representation of the system is shown in Figure 1. Based on this schematic, the governing equations of the system are derived.

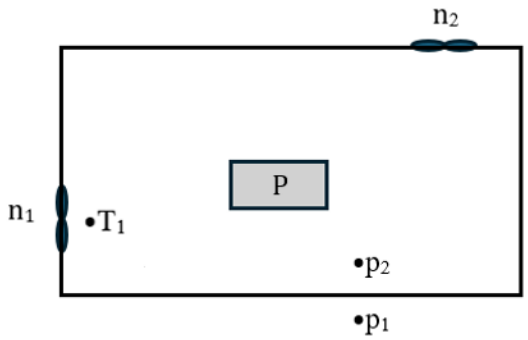


Figure 1. Schematic representation of the cooling system

The developed model represents a simplified data center cooling system. The heating element represents computer servers, which generate heat during operation and act as a disturbance within the system. Since the heat generated by the servers cannot be directly controlled, the cooling system must regulate airflow and temperature in order to maintain stable operating conditions inside the enclosure.

The model includes key system variables such as the fan rotational speeds n_1 and n_2, where n_1 represents the inlet fan speed and n_2 represents the outlet fan speed, the pressure measurements p_1 and p_2, the inlet air temperature T_1, and the power input P of the internal heat source in this case by a resistor. The schematic provides a clear overview of the system components and their interactions, forming the basis for the mathematical modelling approach.

1.1. Pressure Equations

For the purpose of pressure modelling, the ideal gas law (1) is used as a starting point:

$$pV = nRT \quad (1)$$

This equation relates pressure, volume, and temperature of the air inside the system.

Assuming constant volume and negligibly small temperature variations, the pressure dynamics can be approximated using a simplified dynamic model.

The change of pressure depends on the mass balance of air inside the enclosure. The pressure variation is influenced by the difference between the incoming and outgoing air mass flow:

$$\tau_p \frac{d}{dt} \Delta p = -\dot{m}_{leak} + \dot{m}_{air} \quad (2)$$

Where $\dot{m}_{air}$ stands for the net airflow into the system, while $\dot{m}_{leak}$ stands for air leakage due to pressure differences. Furthermore, τ_p represents the pressure time constant of the system, which determines how quickly the pressure responds to changes. The variable Δp represents the pressure difference between the inside of the enclosure and the ambient environment:

$$\Delta p = p_2 - p_1 \quad (3)$$

Where p_2 is the pressure inside the enclosure and p_1 is the pressure outside the enclosure. The net airflow is defined as the difference between inlet and outlet mass flow rates:

$$\dot{m}_{air} = \dot{m}_{in} - \dot{m}_{out} \quad (4)$$

By introducing a linear approximation of leakage proportional to pressure difference, the model can be simplified to a first-order differential equation:

$$\tau_p \frac{d}{dt} \Delta p = -\Delta p + k_p(k_1 n_1 - k_2 n_2) \quad (5)$$

This equation describes how the pressure difference evolves over time depending on the imbalance between the inlet and outlet airflow. The coefficient k_p represents the influence of airflow on pressure dynamics. The coefficients k_1 and k_2 are proportionality constants that relate the fan speeds to the generated airflow. In this model, k_1 corresponds to the inlet fan and k_2 corresponds to the outlet fan, while n_1 and n_2 represent their rotational speeds.

The mass flow rates are defined as:

$$\dot{m}_{in} = k_1 n_1 - k_{\Delta p}\,\Delta p \quad (6)$$

$$\dot{m}_{out} = k_2 n_2 + k_{\Delta p}\,\Delta p \quad (7)$$

These expressions are based on the mass flow depending both on the fan operation and the pressure difference inside the enclosure.

When the inlet fan runs, it increases the internal pressure (6). As a result, the pressure tends to equalize with the surroundings, which reduces the effective inflow. This effect is represented by the negative term proportional to the pressure difference.

In contrast, the outlet mass flow is increased by the pressure difference (7). As the internal pressure rises, the air is pushed out of the enclosure, which increases the outflow rate. Therefore, the pressure term is added in the expression for the outlet mass flow.

The pressure difference influences the airflow by increasing the outflow and reducing the inflow when the internal pressure rises.

For modelling purposes in steady state conditions, the mass balance is satisfied:

$$\dot{m}_{in} = \dot{m}_{out} \quad (8)$$

By equating equations (6) and (7), an expression for the average mass flow rate can be derived, which is later used in the temperature model:

$$\dot{m} = \frac{1}{2}(k_1 n_1 + k_2 n_2) \quad (9)$$

1.2. Temperature Equations

For temperature modelling in this work, the first law of thermodynamics is used in the form of an energy balance for a closed system:

$$\frac{d}{dt}E_{sys} = E_{in} - E_{out} \quad (10)$$

where E_{sys} stands for the total internal energy of the system, $E_{in}$is the energy entering the system, and $E_{out}$is the energy leaving the system.

The internal energy of the system can be expressed as:

$$E = mc\Delta T \quad (11)$$

where m is the mass and c is the specific heat capacity of the material. Since both mass and specific heat capacity are assumed to be constant, the change in internal energy depends only on the temperature change.

For temperature regulation of the enclosed space, it is necessary to consider the main heat flows within the system. These include heat transfer from the resistor to the air (12), heat transfer due to ventilation (16), and possible heat losses to the environment (17).

$$\dot{Q}_{resistor \to room} = kA(T_{resistor} - T_{room}) \quad (12)$$

$$k_{resistor}\frac{d}{dt}T_{resistor} = P - \dot{Q}_{resistor \to room} \quad (13)$$

By substituting the heat transfer term, the resistor temperature equation becomes:

$$k_{resistor}\frac{d}{dt}T_{resistor} = P - kA(T_{resistor} - T_{room}) \quad (14)$$

The room temperature is described by the following energy balance:

$$k_{room}\frac{d}{dt}T_{room} = \dot{Q}_{resistor \to room} + \dot{Q}_{vent} - \dot{Q}_{loss} \quad (15)$$

The ventilation heat flow is defined as:

$$\dot{Q}_{vent} = c_{air}\dot{m}_{in}T_1 - c_{air}\dot{m}_{out}T_{room} \quad (16)$$

The heat loss to the environment is described by:

$$\dot{Q}_{loos} = UA(T_{room} - T_{amb}) \quad (17)$$

By substituting the heat flow expressions into the room temperature balance, the following equation is obtained:

$$k_{room}\frac{dT_{room}}{dt} = kA(T_{resistor} - T_{room}) + c_{air}(\dot{m}_{in}T_1 - \dot{m}_{out}T_{room}) - UA(T_{room} - T_{amb}) \quad (18)$$

Furthermore, $T_{resistor}$ denotes the resistor temperature, while T_{room} represents the air temperature inside the enclosure.

The parameter k represents the heat transfer coefficient between the resistor and the surrounding air, and A represents the effective heat transfer surface area of the resistor. The variables k_{room} and $k_{resistor}$ represent the thermal capacities of the room air and the resistor, respectively.

The variable c_{air} denotes the specific heat capacity of air, while $\dot{m}_{in}$ and $\dot{m}_{out}$ represent the inlet and outlet air mass flow rates. The variable T_1 represents the inlet air temperature, and T_{amb} represents the ambient temperature outside the enclosure. In this model, the inlet air is assumed to originate directly from the surrounding environment; therefore, $T_1 = T_{amb}$.

For the purposes of this project, the inlet and outlet mass flow rates are assumed to be equal $\dot{m}_{in} = \dot{m}_{out} = \dot{m}$ Therefore, the ventilation term can be simplified as:

$$\dot{Q}_{vent} = c_{air}\dot{m}(T_1 - T_{room}) \quad (19)$$

In this model, heat losses through the walls of the enclosure are neglected. The parameter UA represents the overall heat transfer between the system and the environment. Since the enclosure is assumed to be ideally insulated, the heat loss term is set to zero:

$$UA = 0, \ \dot{Q}_{loss} = 0$$

Finally, the simplified room temperature equation used in the simulation is:

$$k_{room}\frac{d}{dt}T_{room} = kA(T_{resistor} - T_{room}) + c_{air}\,\dot{m}\,(T_1 - T_{room}) \quad (20)$$

1.3. Simulink Model

The mathematical model was implemented in the Simulink environment, together with MATLAB, using a block diagram approach. Each differential equation is represented by an integrator block, while algebraic operations are realized using gain and summation blocks.

The inputs of the system include the resistor power and fan speeds, which are modeled using constant and step signals. The outputs are the temperatures and pressure difference.

The Simulink model enables dynamic simulation of the system and provides insight into its behavior under different operating conditions.

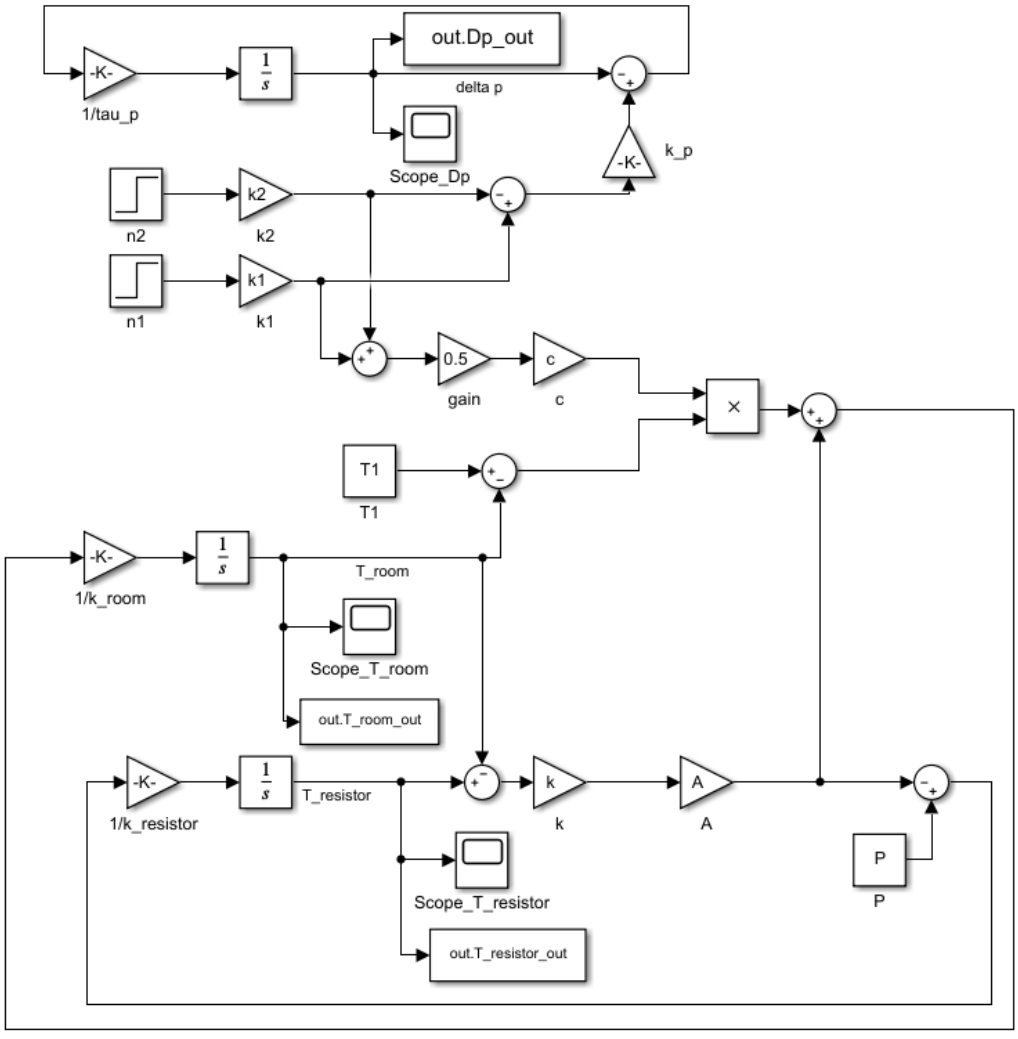


Figure 1.3. Block diagram of the model

1.4. Simulation Results

The simulation results are presented as system responses over time for different operating conditions. The responses analyzed include the pressure difference, room temperature, and resistor temperature.

The simulations were performed by varying the fan speeds, fan step times, and applied heating power. The selected simulation parameters used in the analysis are summarized in Table 1.4., while the corresponding system responses are presented in the following figures. The vertical red lines shown in the figures indicate the moments at which the fan speed setpoints are changed during the simulation.

The purpose of the simulations is to analyze the dynamic behavior of the cooling system under different operating conditions and to evaluate the influence of airflow and heating power on pressure and temperature regulation.

The results confirm that the developed mathematical model successfully captures the main dynamic characteristics of the cooling system. Furthermore, the model provides valuable insight into the expected system behavior and simplifies the development of the physical implementation, in this case the mock-up model.

Table 1.4. Variable values

Figure	1.4.1.	1.4.2.	1.4.3.	1.4.4.	1.4.5.	1.4.6.
$n_{1_step_time}$	20	20	20	20	100	20
$n_{1_initial}$	800	800	800	750	800	800
n_{1_final}	1200	1200	1200	1100	1200	1200
$n_{2_step_time}$	100	40	40	40	20	50
$n_{2_initial}$	800	800	800	800	800	700
n_{2_final}	1200	1200	1200	1200	1200	1000
P	10	10	20	10	10	10

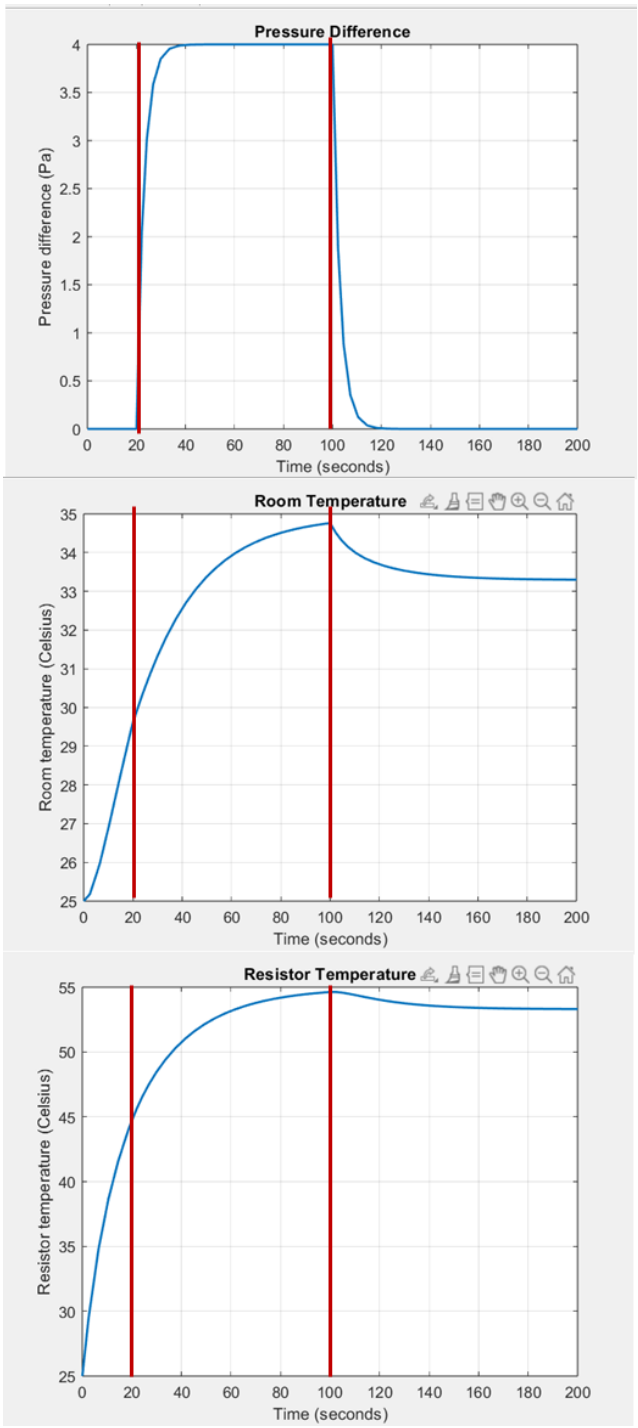


Figure 1.4.1. System response for delayed outlet fan speed increase

The simulation parameters used in this case are summarized in Table 1.4.1, while the corresponding system response is shown in Figure 1.4.1.

Figure 1.4.1 shows the system response when both inlet and outlet fan speeds are increased from 800 rpm to 1200 rpm at different times. The inlet fan speed changes at $t = 20\ s$, while the outlet fan speed changes at $t = 100\ s$. Due to the temporary airflow imbalance, the pressure difference increases and reaches approximately 4 Pa. Once both fans operate at the same speed, the pressure gradually returns to zero. During the interval of positive pressure, the airflow enhances cooling, resulting in a slight decrease in both room and resistor temperatures after $t = 100\ s$.

Table 1.4.1.

$n_{1_step_time}$	20
$n_{1_initial}$	800 rpm
n_{1_final}	1200 rpm
$n_{2_step_time}$	100
$n_{2_initial}$	800 rpm
n_{2_final}	1200 rpm
P	10 W

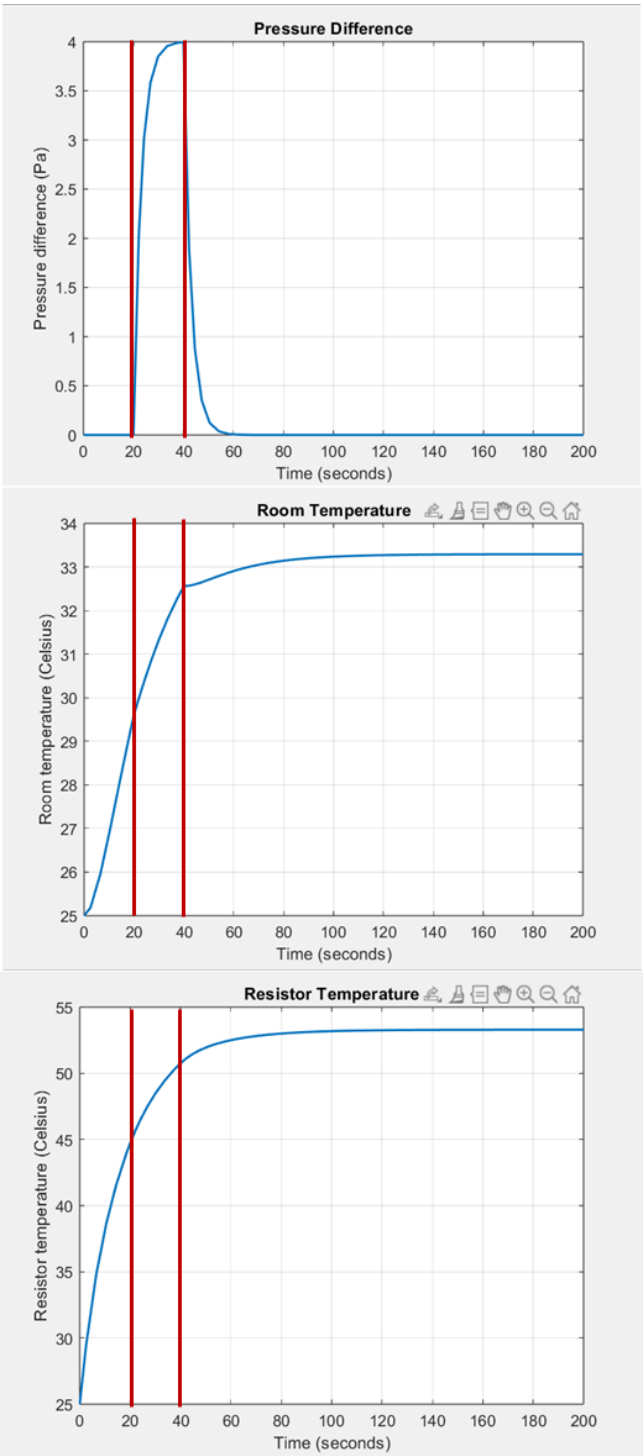


Figure 1.4.2. System response for reduced delay between fan speed changes

The selected simulation parameters are presented in Table 1.4.2, while the obtained responses are shown in Figure 1.4.2.

Figure 1.4.2 presents the system response for a shorter delay between the inlet and outlet fan speed changes. The outlet fan speed is increased at $t = 40\ s$, which causes the pressure difference to return to zero significantly faster compared to Figure 1.4.1.

Since the airflow imbalance exists for a shorter period, the effect on room and resistor temperatures is less pronounced. Both temperatures stabilize smoothly after the fan speeds become equal.

Table 1.4.2.

$n_{1_step_time}$	20
$n_{1_initial}$	800 rpm
n_{1_final}	1200 rpm
$n_{2_step_time}$	40
$n_{2_initial}$	800 rpm
n_{2_final}	1200 rpm
P	10 W

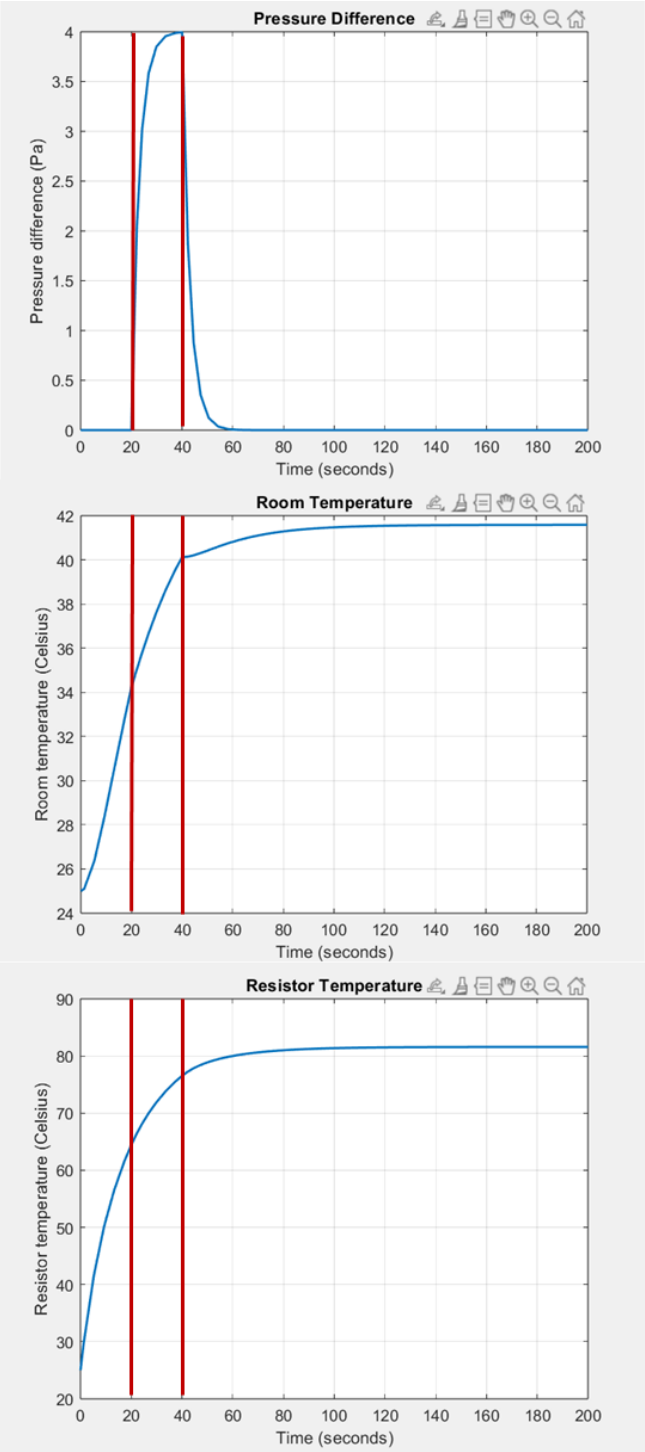


Figure 1.4.3. System response for increased heating power

The simulation parameters for the increased heating power case are summarized in Table 1.4.3, while the corresponding responses are shown in Figure 1.4.3.

Figure 1.4.3 shows the system response for increased heating power $P = 20$. Compared to the previous cases, both room and resistor temperatures reach significantly higher values due to the larger heat input. The resistor temperature rises above 80°C, while the room temperature reaches approximately 42°C. The pressure response remains similar to previous cases since it primarily depends on airflow imbalance rather than heating power.

Table 1.4.3.

$n_{1_step_time}$	20
$n_{1_initial}$	800 rpm
n_{1_final}	1200 rpm
$n_{2_step_time}$	40
$n_{2_initial}$	800 rpm
n_{2_final}	1200 rpm
P	20 W

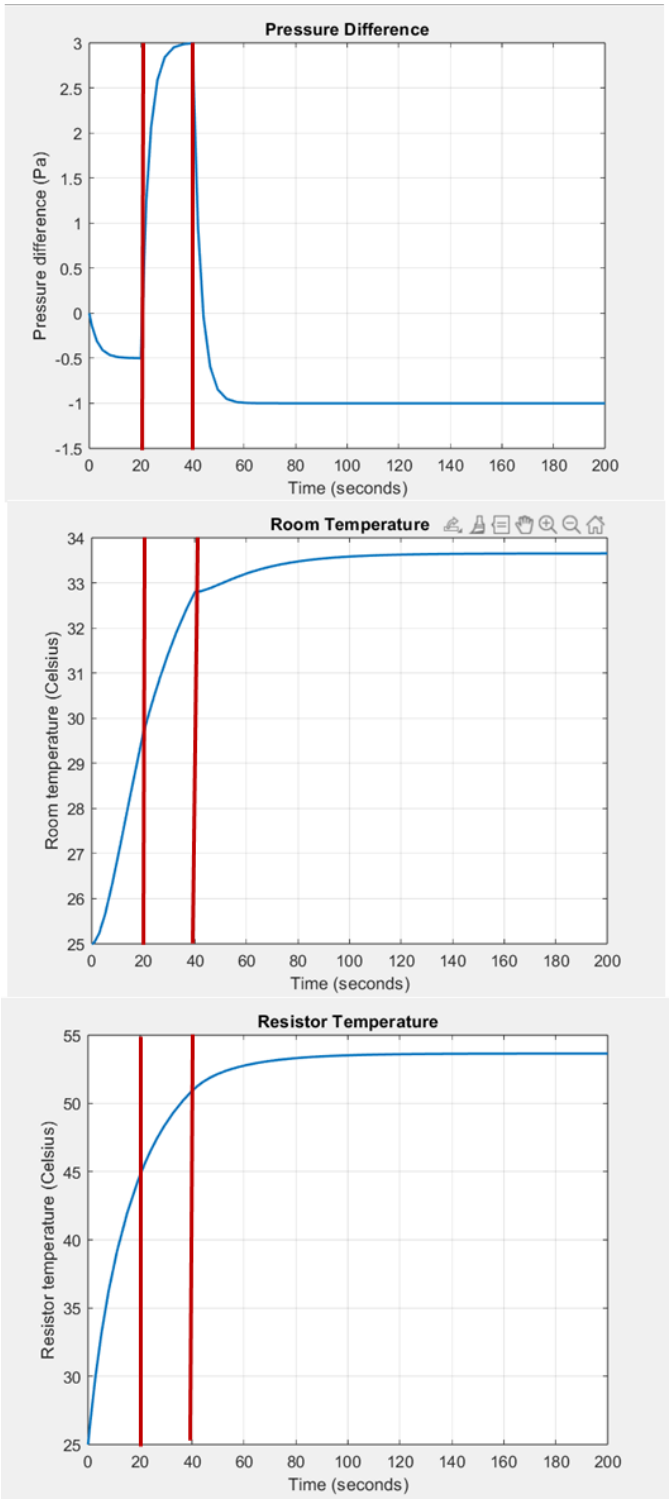


Figure 1.4.4 System response for unequal final fan speeds

The parameters used in this simulation are presented in Table 1.4.4, while the resulting system behavior is shown in Figure 1.4.4.

Figure 1.4.4 illustrates the system behavior when the inlet and outlet fan speeds do not reach the same final values. Since the outlet fan speed remains higher than the inlet fan speed, constant negative pressure is established inside the enclosure. The pressure difference stabilizes around −1Pa, indicating continuous airflow imbalance. The temperature responses remain relatively stable because the airflow conditions do not change significantly after stabilization.

Table 1.4.4.

$n_{1_step_time}$	20
$n_{1_initial}$	750 rpm
n_{1_final}	1100 rpm
$n_{2_step_time}$	40
$n_{2_initial}$	800 rpm
n_{2_final}	1200 rpm
P	10 W

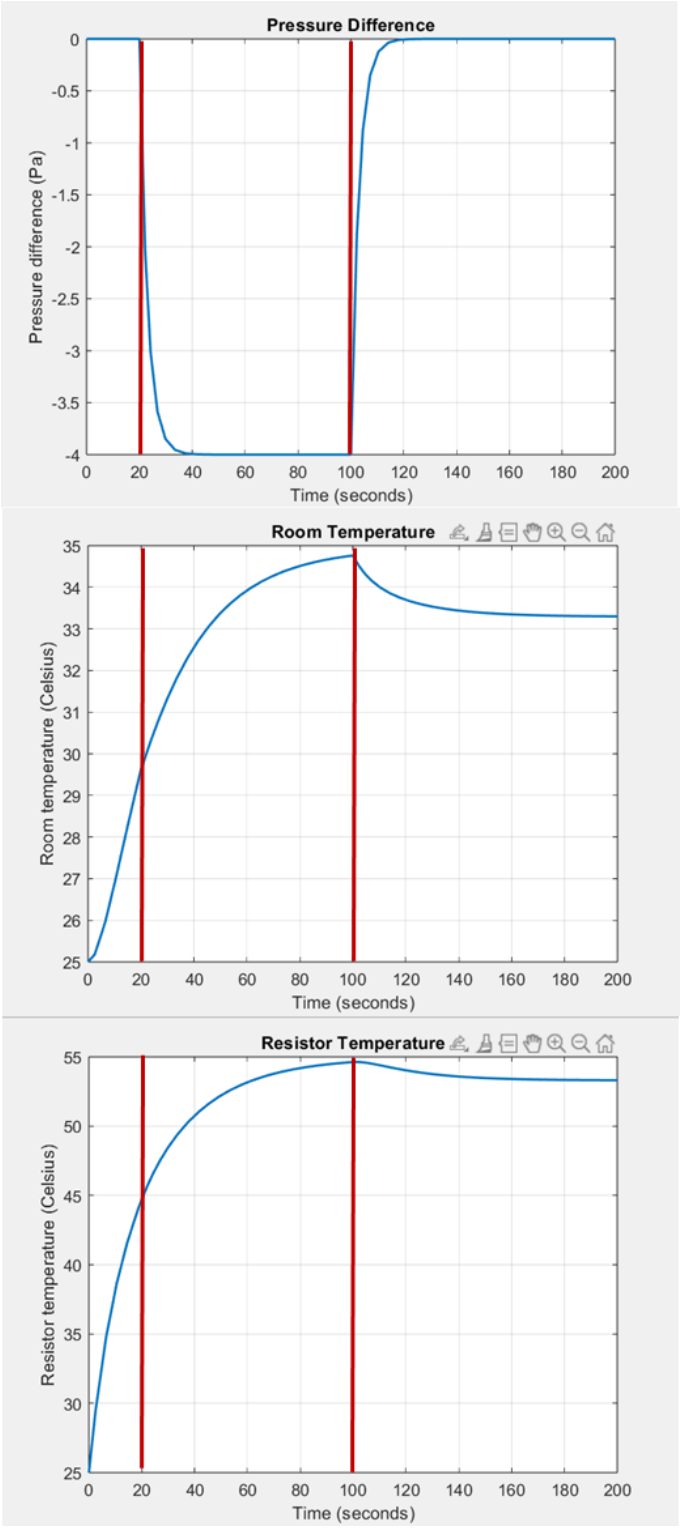


Figure 1.4.5 System response with temporary negative pressure

The selected simulation parameters for this case are summarized in Table 1.4.5, while the corresponding responses are shown in Figure 1.4.5.

Figure 1.4.5 shows the pressure response when the outlet fan speed increases before the inlet fan speed. This causes temporary negative pressure inside the enclosure, reaching approximately -4Pa. After the inlet fan speed is increased at $t = 100\ s$, the airflow becomes balanced and the pressure gradually returns to zero. The increased airflow also causes a slight decrease in room and resistor temperatures after the second fan speed change.

.

Table 1.4.5.

$n_{1_step_time}$	100
$n_{1_initial}$	800 rpm
n_{1_final}	1200 rpm
$n_{2_step_time}$	20
$n_{2_initial}$	800 rpm
n_{2_final}	1200 rpm
P	10 W

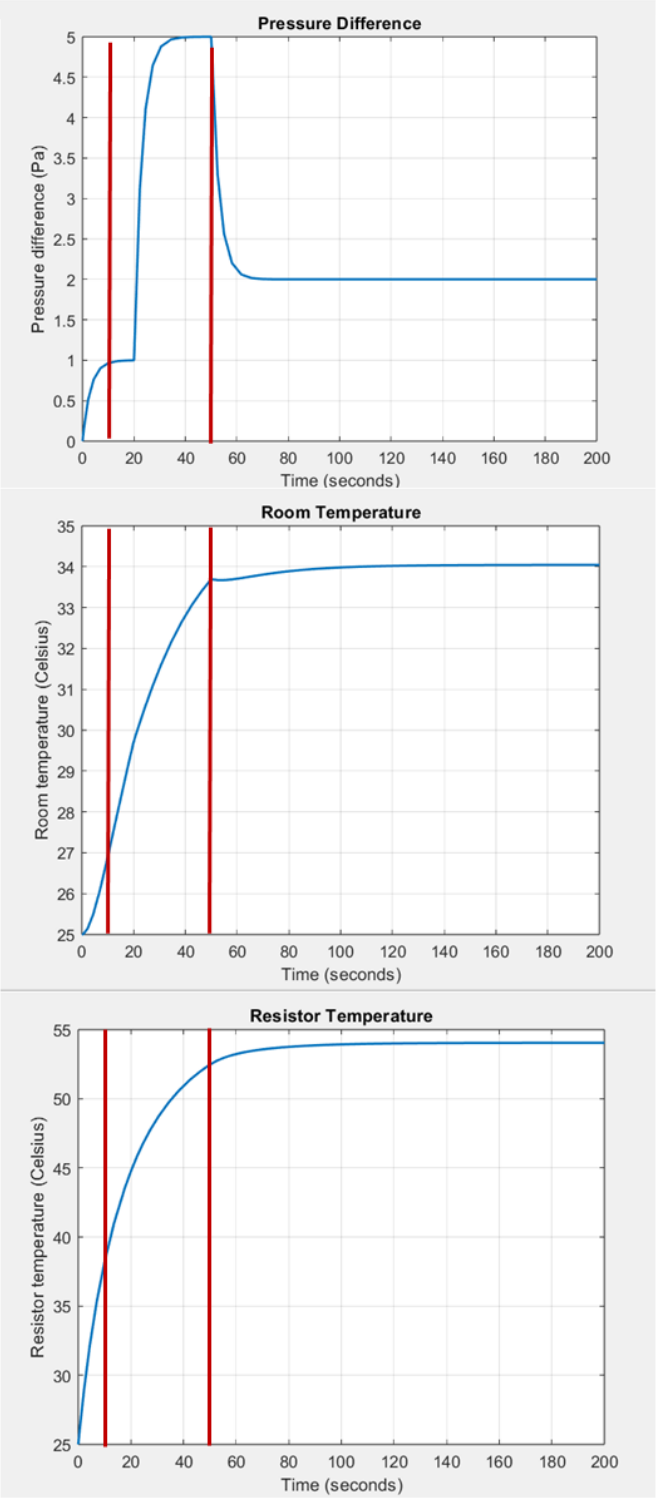


Figure 1.4.6 System response with positive steady-state pressure

The simulation parameters used for this case are presented in Table 1.4.6, while the corresponding responses are shown in Figure 1.4.6.

Figure 1.4.6 presents the case where the inlet fan speed remains higher than the outlet fan speed after stabilization. As a result, the system maintains a constant positive pressure of approximately 2 Pa. The room and resistor temperatures stabilize at values similar to the previous simulations, while the pressure response confirms the influence of unequal fan speeds on the internal pressure conditions.

Table 1.4.6.

$n_{1_step_time}$	20
$n_{1_initial}$	800 rpm
n_{1_final}	1200 rpm
$n_{2_step_time}$	50
$n_{2_initial}$	700 rpm
n_{2_final}	1000 rpm
P	10 W

2. Mock-Up of the Cooling System

In this work, a mock-up model of the cooling system was developed to approximate the operation of a cooling system in a data center environment. The aim of the mock-up was not to accurately simulate the detailed interaction between hot and cold air, but to provide a simplified representation of the cooling process and evaluate the performance of the control system.

The system consists of two distinct thermal zones: a warmer region near the heat source and a cooler region influenced by incoming air. The heat source is represented by a resistor, which simulates the behavior of computer equipment that generates heat during operation.

The system is equipped with temperature and pressure sensors, which are used to monitor the internal conditions and enable automated control of the cooling process. Based on the measured values, the control system adjusts the fan operation to maintain desired conditions inside the enclosure.

To approximate real data center conditions, where hot and cold air regions are separated, the airflow inside the enclosure is arranged accordingly. Cooler air is introduced at the lower part of the enclosure, while warmer air is extracted from the upper part. This configuration enables partial separation of hot and cold air and mimics the airflow management used in real data centers.

The physical appearance of the mock-up model is shown in Figure 2.

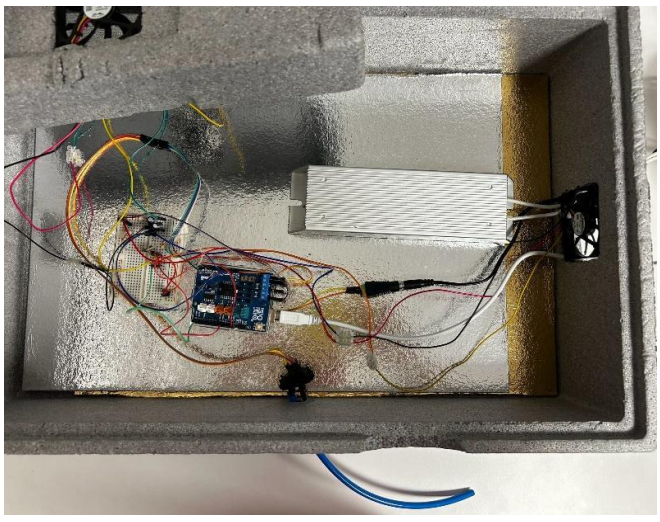


Figure 2. Mock-up model

2.1. System Components

The mock-up system consists of several key components, each with a specific role in the operation of the cooling system. The main components are described below.

2.1.1. Controller and Motor Shield

The Arduino Uno microcontroller serves as the main control unit of the system and is the core of the control loop. It is responsible for acquiring sensor data, executing the control algorithm, and generating output signals for system actuation. Additionally, it enables communication with external software for data processing and visualization.

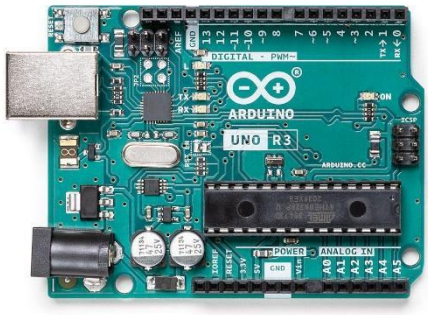


Figure 2.1.1.1. Arduino uno [2]

Since the Arduino cannot directly supply the current required by actuators such as fans and the heating element, a motor shield is used. This shield acts as a high-current interface, based on the L298P dual full-bridge driver, allowing safe control of the actuators. It translates low-voltage control signals into high-power PWM outputs and is powered by an external supply.

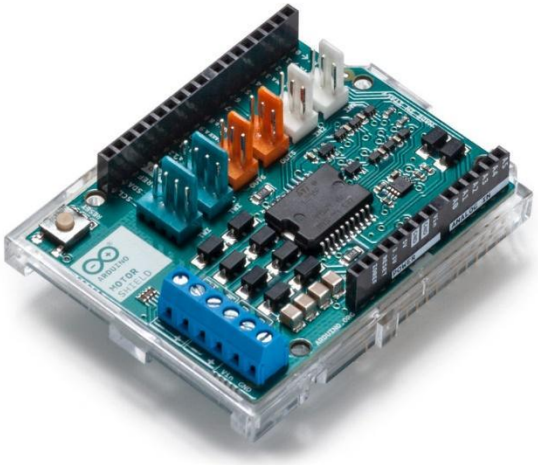


Figure 2.1.1.2. Motor Shield [3]

2.1.2. Sensors

The system uses temperature and pressure sensors to provide feedback for control.

Temperature Sensor

The LM35 CZ is a precision temperature sensor that provides an output voltage linearly proportional to the temperature in degrees Celsius. The sensor operates in the temperature range from $-55°C$ to $150°C$, where an output of $10\,mV$ corresponds to $1°C$. It is powered by a $5\,V$ supply voltage and enables accurate real-time temperature monitoring without requiring additional calibration. The sensor output is read through the Arduino analog-to-digital converter for temperature calculation [4].



Figure 2.1.2.1. Temperature sensor [5]

Pressure Sensor

The SDP610-025PA Sensirion sensor is a digital differential pressure sensor used to measure small pressure differences between the inside of the enclosure and the ambient environment. The sensor measures pressure in the range of $\pm 25\,Pa$ and communicates with the Arduino using the I2C interface through the SDA and SCL communication lines. The module operates at a supply voltage of $3.3\,V$ and provides accurate measurements of airflow-related pressure variations [6].



Figure 2.1.2.2. Pressure sensor [7]

2.1.3. Actuators

The actuators in the system include a heating element and fans, which directly influence the thermal and flow conditions inside the enclosure.

Heating Element

The heating element consists of an RXLG dynamic braking resistor with a rated power of $300\,W$ and an internal resistance of $2.2\,\Omega$. The resistor is mounted in an aluminum housing with fins that increase the effective heat transfer surface area and improve heat dissipation to the surrounding air. It is powered using an external $12\,V$ supply and simulates the thermal load generated by computer servers inside a data center [8].



Figure 2.1.3.1. Heating element [9]

Fans

The cooling system uses two SUNON MF60101V1-1000U-G99 DC fans mounted on opposite sides of the enclosure to regulate airflow and temperature. The fans operate at a nominal voltage of $12\,V$ and include an integrated tachometer output that generates two pulses per revolution, enabling rotational speed measurement. The fans are controlled using Pulse Width Modulation (PWM), which allows precise airflow regulation for temperature and pressure control. One fan is primarily used for air intake, while the other is used for air exhaust, enabling partial separation of warm and cold air regions inside the enclosure [10].



Figure 2.1.3.2. Fan [11]

2.1.4. Additional Electronic Components

The system also includes passive electronic components such as resistors, capacitors, and inductors. These elements are used for filtering, signal conditioning, and stabilization of the electrical circuits, ensuring reliable operation of sensors and actuators.

Resistors are used to limit current and protect electronic components, while capacitors help reduce noise and smooth voltage fluctuations. Inductors contribute to filtering and improving the stability of the power supply and control signals.

2.2. System Configuration and Wiring Diagram

The connection of all system components is shown in the wiring diagram in Figure 2.2. The diagram illustrates how the sensors, actuators, and control units are interconnected.

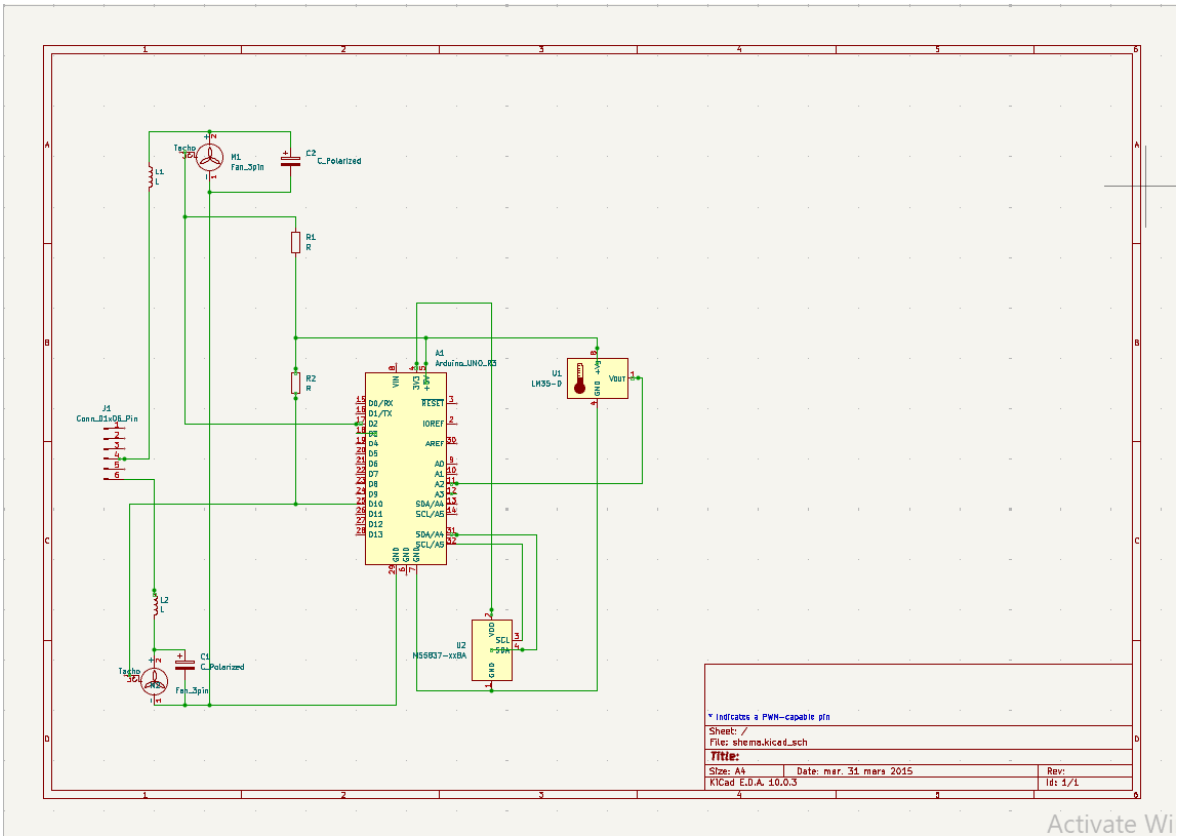


Figure 2.2. Connection diagram

2.3. Control System Design

The control system is designed to regulate airflow, pressure, and temperature inside the enclosure. It is based on a closed-loop control structure, where measured values are used as feedback to adjust the operation of the actuators.

The control strategy is implemented in several stages, including speed control, pressure control, and temperature control, which are described in the following sections.

The overall control system is structured around the speed control loop as the inner loop, while pressure and temperature control act as outer loops that set the reference speed. In this way, both pressure and temperature regulation are achieved by adjusting the fan speed.

2.3.1. Speed Control

At the initial stage of the implementation, accurate speed measurement was not possible due to unstable tachometer signals. The Arduino was unable to correctly detect the impulses generated by the fan, which resulted in incorrect speed readings.

To verify the signal quality, an oscilloscope was used as a reference measurement tool. The oscilloscope displayed the correct signal, while the Arduino measurements were inaccurate.

To resolve this issue, a hardware signal conditioning circuit was introduced. The circuit consisted of a capacitor and an inductor, which acted as a filter to stabilize the signal and improve impulse detection.

Once reliable speed measurement was achieved, the speed control loop was implemented. The control system compares the reference speed with the measured speed and generates an error signal.

This error is processed by a PI controller. The controller’s output is then limited to a predefined range before being converted into a PWM signal that drives the fan. The PWM signal is constrained between minimum and maximum values, where the maximum value is 255, while values below 60 are avoided due to unstable fan operation.

The block diagram of the speed control system is shown in Figure 2.3.1.

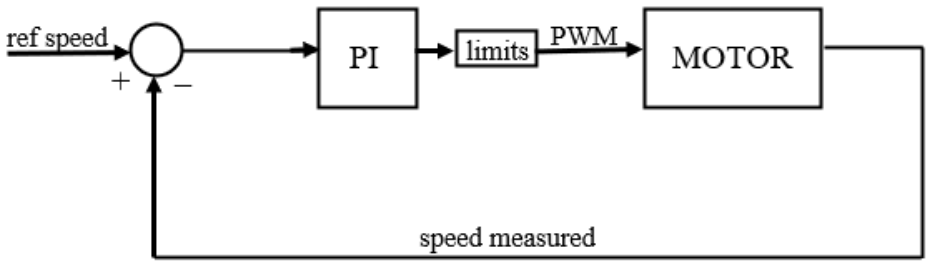


Figure 2.3.1. Block diagram of the speed control system

2.3.2. Pressure Control

The pressure control is implemented as an outer control loop. The reference pressure is compared with the measured pressure, and the resulting error is processed by a PI controller.

The output of the pressure controller defines the reference speed for the inner speed control loop. In this way, a cascade control structure is formed.

The controller's output is limited to ensure that the fan speed stays within its physical constraints. The maximum allowable fan speed is 4000 rpm, while the minimum speed is limited to 1700 rpm. This lower limit corresponds to a PWM value of approximately 60, below which the fan operation becomes unstable. This limit is applied before passing the reference signal to the inner control loop.

The measured pressure is fed back to the input, forming a closed-loop system.

The block diagram of the pressure control system is shown in Figure 2.3.2.

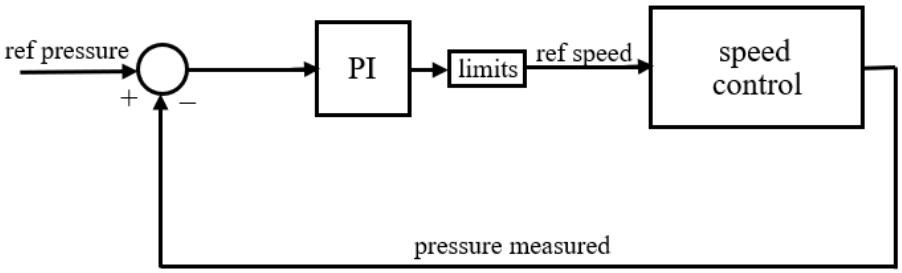


Figure 2.3.2. Block diagram of the pressure control system

2.3.3. Temperature Control

In addition to pressure control, temperature regulation was also implemented. The aim of the temperature control loop is to keep a constant temperature inside the enclosure.

The temperature is measured using sensors, and the measured value is compared with the reference temperature. Based on the resulting error, the control system adjusts the fan speed to regulate the cooling process.

When the temperature inside the enclosure increases above the desired value, the fan speed increases to enhance cooling. Conversely, when the temperature is within the desired range, the fan speed is reduced, allowing the system to maintain stable thermal conditions.

The control structure is based on feedback from temperature sensors and proper adjustment of actuator signals.

2.3.4. PID Controller

A Proportional-Integral controller (PI) was selected instead of a Proportional-Integral-Derivative controller (PID) due to the characteristics of the system. The derivative component in a PID controller is highly sensitive to noise, which is present in the measured signals.

Since the system exhibits relatively slow dynamics and does not require fast transient response, the proportional-integral controller provides sufficient performance.

Additionally, the PI controller is simpler to implement and tune, making it more suitable for this application.

2.4. Software Implementation

The control system was implemented using both Arduino and MATLAB environments. Arduino was used for real-time control, data acquisition, and actuator management, while MATLAB was used for data processing, visualization, and parameter tuning.

The complete Arduino and MATLAB source codes are provided in Appendix A and Appendix B.

Figure 2.4. shows the communication structure between the Arduino controller and the MATLAB environment, including the exchanged measurement and control signals.

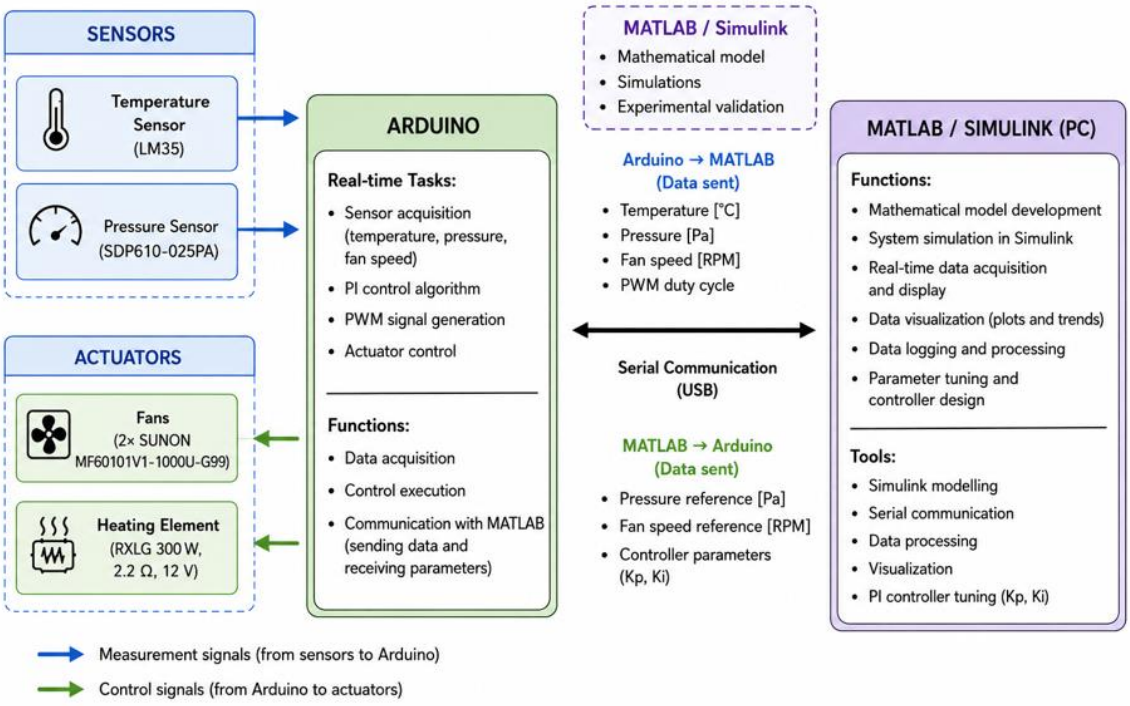


Figure 2.4. Communication structure between Arduino and MATLAB

3. Results and Comparison

This section presents the results obtained from both the MATLAB/Simulink simulation and the physical mock-up model. The system's behavior is analyzed under different operating conditions, and the results are compared in order to evaluate the performance of the implemented control system.

3.1. Speed Control

To achieve satisfactory system performance, it was first necessary to properly tune the PI controller. The initial controller parameters were set to $K_p = 0.04$ and $K_i = 0.02$, the corresponding response is shown in Figure 3.1.1.

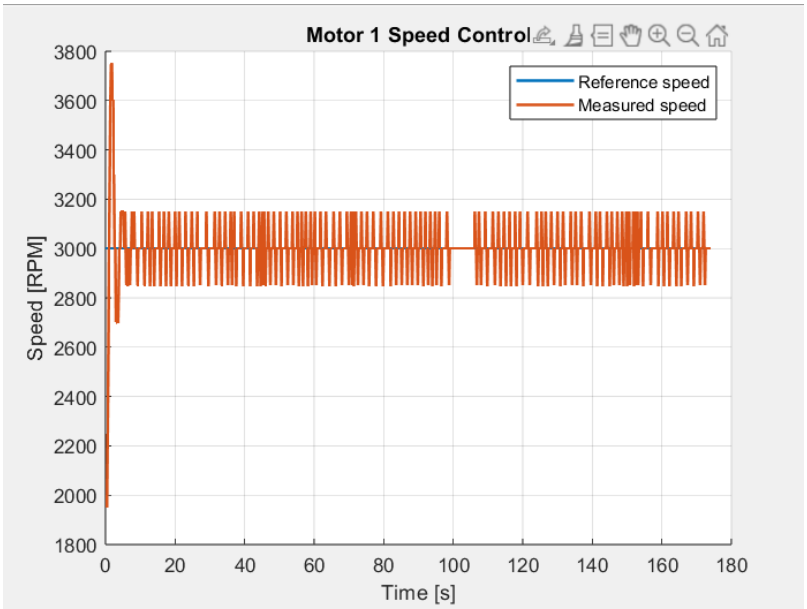


Figure 3.1.1. Kp = 0.04, Ki = 0.02;

The response obtained was not satisfactory, indicating the need for further adjustment of the controller parameters. Therefore, the parameters were tuned iteratively until a more stable and accurate response was achieved, as shown in Figure 3.1.2.

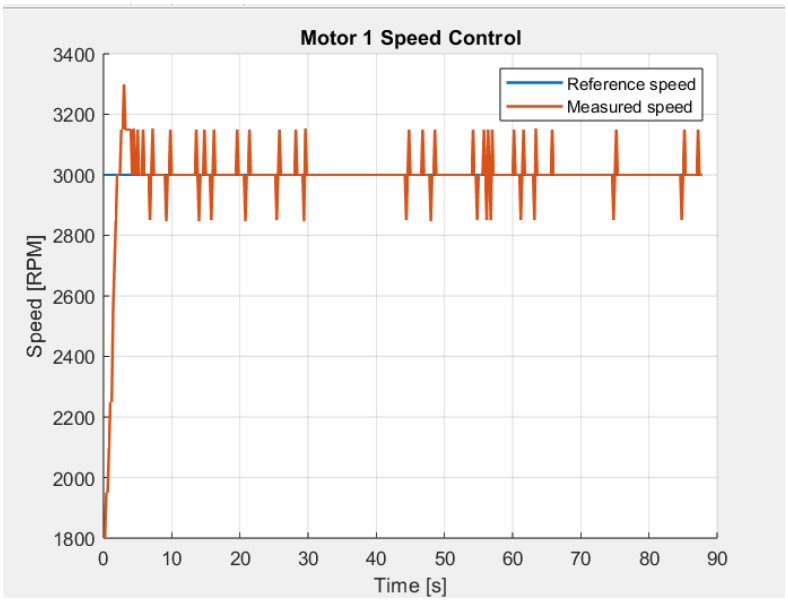


Figure 3.1.2. Kp = 0.015, Ki = 0.001

Finally, the system's behavior was analyzed for changes in reference speed. The results, shown in Figure 3.1.3, confirm that the control system is able to track the reference signal with acceptable accuracy and stability.

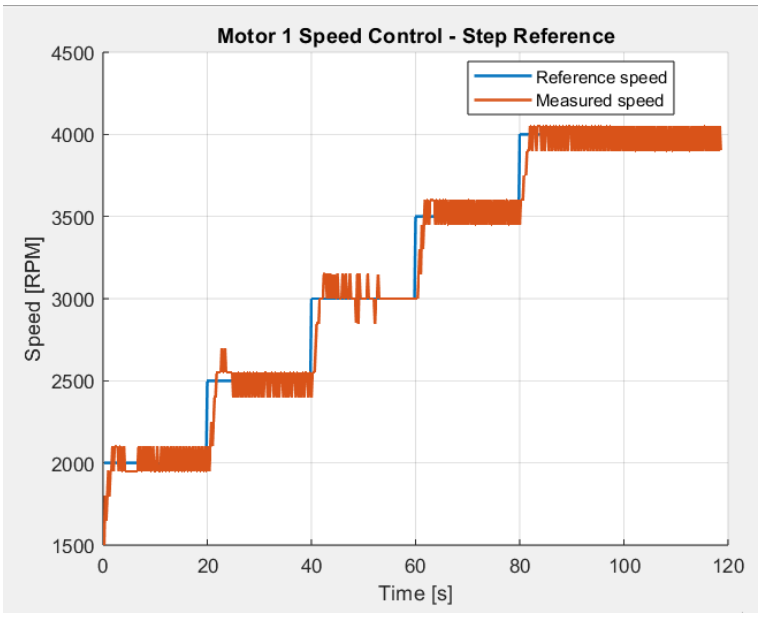


Figure 3.1.3. Speed control

3.2. Pressure Response Analysis

To analyze the pressure behavior of the system, the inlet and outlet fan speeds were varied under controlled conditions in both the mathematical model and the physical mock-up system.

In the MATLAB/Simulink model, the outlet fan speed $n_2$was increased stepwise from 2000 rpm to 4000 rpm at intervals of 30 seconds, while the inlet fan speed $n_1$was kept constant. The corresponding simulated pressure response is shown in Figure 3.2.1.

In the second simulation, the roles of the fans were reversed. The inlet fan speed $n_1$was varied, while the outlet fan speed $n_2$remained constant. The obtained simulated response is shown in Figure 3.2.2.

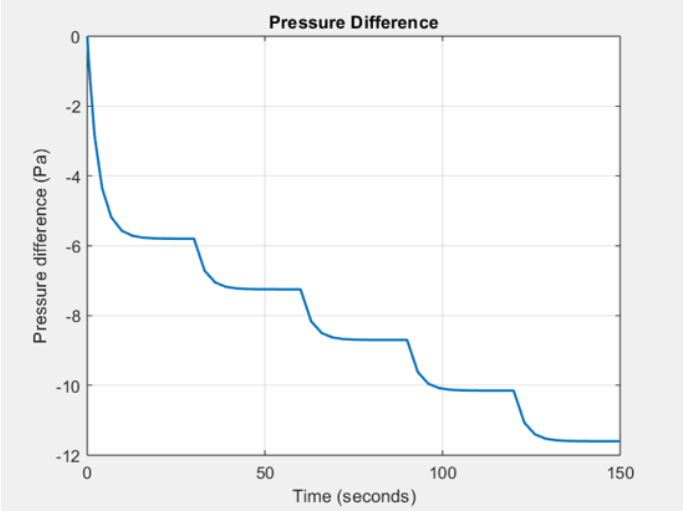


Figure 3.2.1. Mathematical model of pressure response

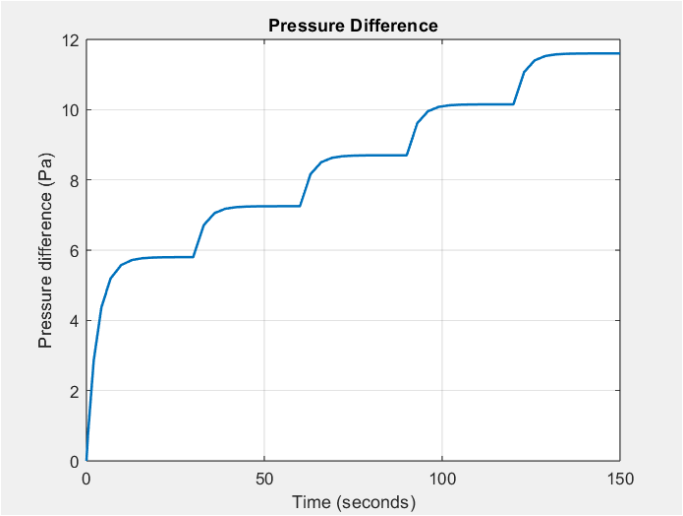


Figure 3.2.2. Mathematical model of pressure response

The simulation results show that the pressure difference depends on the imbalance between inlet and outlet airflow. Increasing the outlet fan speed results in negative pressure inside the enclosure, while increasing the inlet fan speed leads to positive pressure.

The corresponding experimental responses obtained from the physical mock-up model are shown in Figures 3.2.3 and 3.2.4.

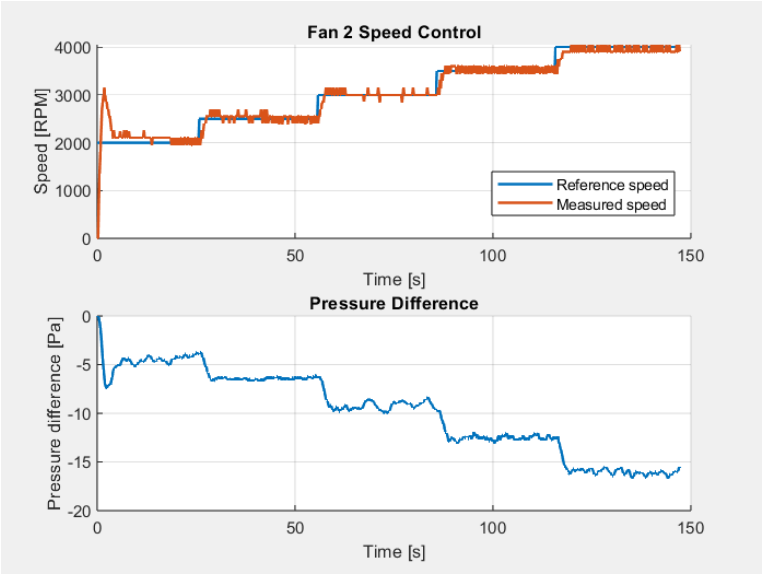


Figure 3.2.3. Mock-up model of pressure difference

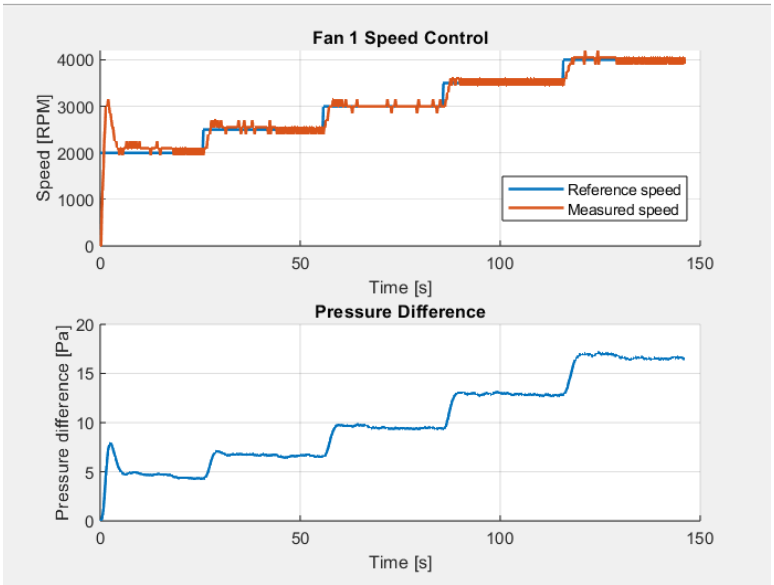


Figure 3.2.4. Mock-up model of pressure difference

Although the experimental results do not exactly match the simulation results, a similar trend can be observed. These differences are expected due to real-world effects such as sensor accuracy, airflow disturbances, and system nonlinearities. However, the overall agreement between the simulation and the experimental results confirms the validity of the developed mathematical model.

3.3. Temperature Control Analysis

The temperature control behavior was analyzed by varying the fan speed in the simulation model. In the Simulink environment, the fan speed increased stepwise every 100 seconds, from 2000 rpm to 4000 rpm. The results show a decrease in temperature with increasing fan speed, which is expected due to enhanced cooling, as shown in Figure 3.3.1.

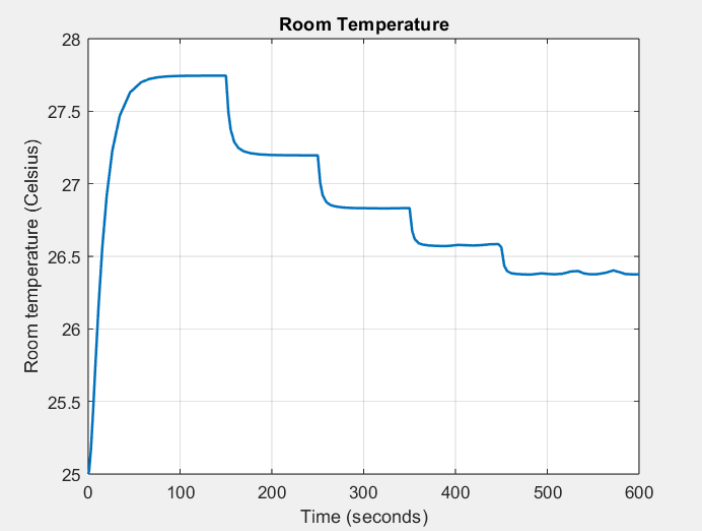


Figure 3.3.1. Mathematical model

In the physical mock-up model, the heating element was first used to increase the temperature inside the enclosure, after which the cooling process was analyzed. The corresponding experimental results are shown in Figure 3.3.2.

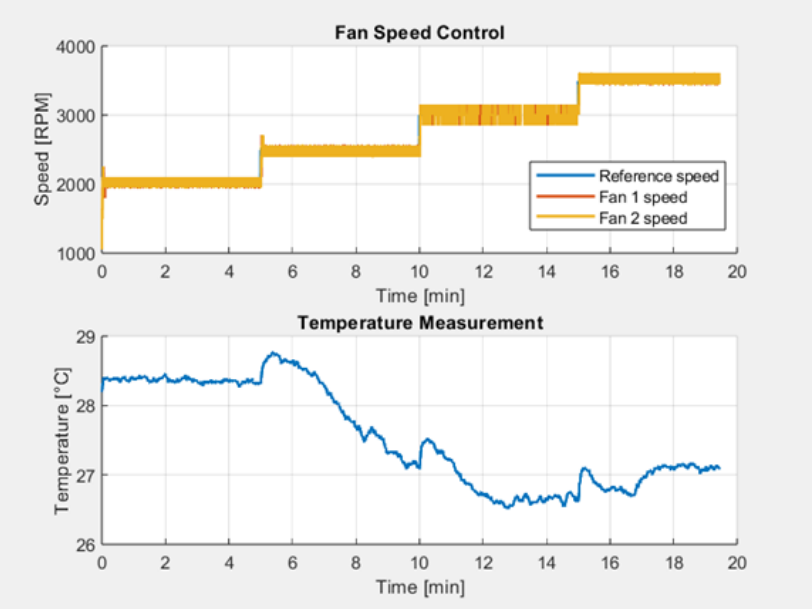


Figure 3.3.2. Mock-up model

The fan speed increased stepwise approximately every 5 minutes, from 2000 rpm to 3500 rpm. As the fan speed increased, the temperature inside the enclosure gradually decreased due to the enhanced airflow and cooling effect.

A small temporary increase in temperature can be observed after approximately 5 minutes. This effect is likely caused by additional heat generated by the electronic components inside the enclosure, such as the Arduino board, motor shield, and other control electronics, before the increased airflow produces a noticeable cooling effect.

At lower fan speeds, the temperature reduction was more noticeable. However, at higher fan speeds, the temperature changes became significantly smaller, and the system gradually approached a steady-state value of approximately 27∘C. This behavior indicates that the cooling capability of the system becomes limited by the surrounding ambient conditions.

Small oscillations and measurement disturbances can also be observed in the experimental results, which are expected in a real physical system due to sensor noise, airflow disturbances, and environmental influences. Nevertheless, the overall trend of decreasing temperature with increasing fan speed is consistent with the expected physical behavior of the cooling system.

3.4. Integrated System Performance

After analyzing the individual control loops, the complete control system was implemented and tested. The system integrates speed, pressure, and temperature control, where the fan speed is adjusted based on feedback from both pressure and temperature measurements.

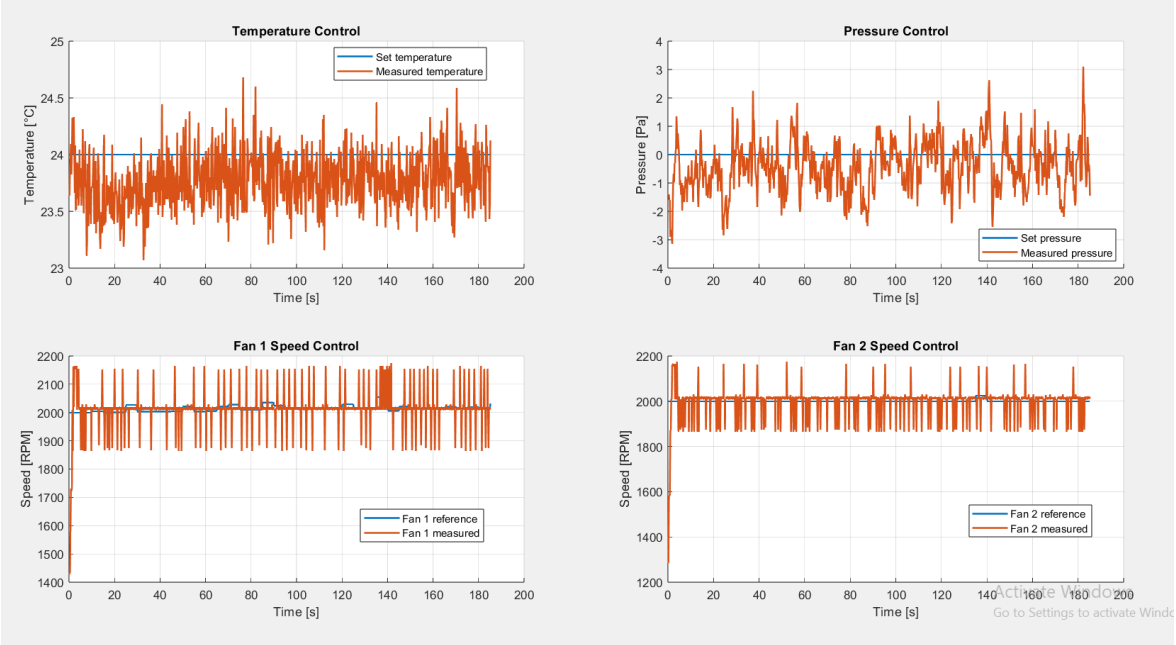


Figure 3.4. Automated control system

The obtained results are presented through several system responses, including temperature tracking, fan speeds, and pressure regulation, as shown in Figure 3.4. The temperature response shows that the system can follow the reference temperature with satisfactory accuracy.

The fan speed responses for both fans show good tracking between the reference and measured values, demonstrating proper operation of the inner speed control loop. Additionally, the pressure response shows that the system can keep the desired pressure level with acceptable deviations.

The results confirm that the system is capable of automatically responding to changes in operating conditions and keeping stable behavior. The control system successfully regulates airflow to influence both pressure and temperature inside the enclosure.

Although some differences between the simulation and the mock-up model were observed, the overall system behavior stays consistent with the expected dynamics.

This demonstrates that the proposed control strategy is effective and suitable for simplified thermal management applications.

Conclusion

In this work, a control system for thermal processes in a data center environment was developed and analyzed using both a mathematical model and a physical mock-up model. The system was designed to regulate temperature and pressure by adjusting the fan speed based on feedback signals.

A mathematical model based on thermodynamic principles was implemented and simulated in the MATLAB/Simulink environment. In addition, a physical mock-up model was constructed to validate the system behavior under real conditions.

The results show that the control system can maintain stable operation and respond to changes in operating conditions. The implemented PI controllers provided satisfactory performance for speed, pressure, and temperature regulation.

Differences between the simulation and experimental results were observed, particularly in temperature measurements, due to real-world effects such as sensor placement and heat sources. However, the overall system's behavior stays consistent with the expected dynamics.

The developed system provides a simplified but effective representation of thermal control in data center environments and demonstrates the importance of proper control design and system implementation.

References

[1] B. Halasz, Uvod u termodinamiku. Zagreb, Croatia: Faculty of Mechanical Engineering and Naval Architecture, University of Zagreb, 2021.

[2] “Arduino Uno,” Arduino, Available: https://store-usa.arduino.cc/cdn/shop/files/A000066_03.front_934x700.jpg?v=1727102657 Accessed: May 24, 2026.

[3]“Motor Shield“, Available: https://store.arduino.cc/cdn/shop/files/A000079_01.iso_884x700.jpg?v=1727098553 Accessed: May 24, 2026.

[4]”LM35”Datasheet. Available: https://www.ti.com/lit/ds/symlink/lm35.pdf Accessed: May 28, 2026.

[5]“Temperature sensor“, Available: https://m.media-amazon.com/images/I/51ZgsQ3Vp7L._AC_UF1000,1000_QL80_.jpg Accessed: May 24, 2026.

[6] “SDP600 Series” Datasheet. Available:https://sensirion.com/media/documents/63859DD0/6166CF0E/Sensirion_Differential_Pressure_Datasheet_SDP600Series.pdf Accessed: May 28, 2026.

[7]“Pressure sensor“, Available: https://gr.mouser.com/images/sensirion/lrg/SDP610_SPL.jpg Accessed: May 24, 2026.

[8]”RXLG Braking Resistor” Datasheet. Available: http://www.twjohm.com/Data/twjohm/upload/file/20210828/RXLG-Braking%20Resistor.pdf Accessed: May 28, 2026.

[9]]“Heating element“, Available: https://www.alibaba.com/product-detail/RXLG-100W-1000W-Aluminum-Shell-500_60797305674.html Accessed: May 24, 2026.

[10]”Sunon fan” Datasheet. Available: https://www.sunon.com/en/MANAGE/Docs/PRODUCT/286/360/Sunon%20DC%20Brushless%20Fan%20&%20Blower_(240-A).pdf Accessed: May 28, 2026.

[11]“Fan“, Available: https://cdn.radioparts.com.au/images/ProductImages/12110599_v2.jpg Accessed: May 24, 2026.

Abstract

This thesis presents the development and analysis of a control system for thermal processes in a data center environment using a simplified mock-up model. The aim is to design a control strategy capable of regulating temperature and pressure by adjusting airflow within an enclosed space.

A mathematical model based on thermodynamic principles is developed and implemented in the MATLAB/Simulink environment. In addition, a physical mock-up model is constructed to validate the system's behavior under real operating conditions.

The control system is based on a closed-loop structure with a PI controller, where fan speed is adjusted according to feedback from pressure and temperature sensors. The results show that the system can maintain stable operation and respond to changes in operating conditions.

Differences between simulation and experimental results are observed due to real-world effects, but the overall system behavior is consistent with the expected dynamics. The proposed approach provides a simplified but effective representation of thermal control in data center environments.

Summary

In this work, a control system for thermal processes in a data center environment was developed using both simulation and experimental approaches. A mathematical model of the system was created based on energy balance and pressure equations and implemented in MATLAB/Simulink.

A physical mock-up model was also constructed to represent a simplified data center cooling system, where a resistor simulates heat generation and fans are used to regulate airflow. The system includes temperature and pressure sensors, which provide feedback for control.

The control strategy is based on PI controllers, where fan speed is used as the main control variable. The system was analyzed through speed, pressure, and temperature control, and the performance was evaluated under different operating conditions.

The results show that the system can successfully regulate the internal conditions, although differences between the model and the real system were observed. These differences are mainly caused by practical factors such as sensor placement and additional heat sources.

Overall, the developed system demonstrates a simplified but effective approach to thermal management and highlights the importance of control system design in real applications.

Abbreviations

ref	reference
PWM	Pulse Width Modulation
PI	Proportional-Integral controller

Appendix A – Arduino Implementation

```
#include <EnableInterrupt.h>
#include <Wire.h>

#define LM35_PIN A2
#define SDP610_ADDR 0x40

// TACH pins
const byte fan1TachPin = 2;
const byte fan2TachPin = 10;

// Motor Shield pins
const byte fan1PwmPin = 3;
const byte fan2PwmPin = 11;
const byte fan1DirPin = 12;
const byte fan2DirPin = 13;
const byte fan1BrakePin = 9;
const byte fan2BrakePin = 8;

volatile unsigned long pulseCount1 = 0;
volatile unsigned long pulseCount2 = 0;
volatile unsigned long lastPulse1 = 0;
volatile unsigned long lastPulse2 = 0;

const int pulsesPerRev = 2;

// OUTER LOOP setpoints
float T_SET = 24.0;   // desired temperature
float P_SET = 0.0;    // desired pressure
```

```
float T_BAND = 0.3;
float P_BAND = 0.5;

// OUTER LOOP PI gains
float KpT = 25.0;
float KiT = 0.02;
float KpP = 0.8;
float KiP = 0.02;

// INNER LOOP speed PI gains
float KpSpeed = 0.015;
float KiSpeed = 0.001;

// RPM limits
float rpmMin = 2000.0;
float rpmMax = 4000.0;

// PWM limits
const int pwmMin = 60;
const int pwmMax = 255;

int pwm1 = 60;
int pwm2 = 60;

// Integrators
float iT = 0.0;
float iP = 0.0;
float iSpeed1 = 0.0;
float iSpeed2 = 0.0;
```

```
// Filters
float alphaT = 0.2;
float alphaP = 0.2;
float Tf = NAN;
float Pf = NAN;

// Timing
const unsigned long speedLoopMs = 200;
const unsigned long outerLoopMs = 1000;

unsigned long lastSpeedMs = 0;
unsigned long lastOuterMs = 0;

float rpmRefBase = 2000.0;
float rpmRef1 = 2000.0;
float rpmRef2 = 2000.0;

// Tach filter
const unsigned long minPulseGapMicros = 4000;
const unsigned long maxPulseGapMicros = 300000;

void fan1ISR() {
  unsigned long now = micros();

  if (lastPulse1 == 0) {
    lastPulse1 = now;
    return;
  }
```

```
  unsigned long gap = now - lastPulse1;

  if (gap > minPulseGapMicros && gap < maxPulseGapMicros) {
    pulseCount1++;
    lastPulse1 = now;
  }
}

void fan2ISR() {
  unsigned long now = micros();

  if (lastPulse2 == 0) {
    lastPulse2 = now;
    return;
  }

  unsigned long gap = now - lastPulse2;

  if (gap > minPulseGapMicros && gap < maxPulseGapMicros) {
    pulseCount2++;
    lastPulse2 = now;
  }
}

float deadbandError(float value, float setpoint, float band) {
  float e = value - setpoint;

  if (e > band) return e - band;
  if (e < -band) return e + band;
```

```
  return 0.0;
}

float readTemperatureLM35() {
  int raw = analogRead(LM35_PIN);
  float voltage = raw * (5.0 / 1023.0);
  float tempC = voltage * 100.0;
  return tempC;
}

float readPressureSDP610() {
  Wire.beginTransmission(SDP610_ADDR);
  Wire.write(0xF1);
  Wire.endTransmission();

  delay(10);

  Wire.requestFrom(SDP610_ADDR, 6);

  if (Wire.available() == 6) {
    int16_t rawPressure = (Wire.read() << 8) | Wire.read();

    Wire.read(); // CRC
    Wire.read(); // temp MSB
    Wire.read(); // temp LSB
    Wire.read(); // CRC

    float pressurePa = rawPressure / 60.0;
    pressurePa = constrain(pressurePa, -25.0, 25.0);
```

```
    return pressurePa;
  }

  return -9999.0;
}

void readSerialCommands() {
  while (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();

    if (cmd.startsWith("SPT:")) {
      T_SET = cmd.substring(4).toFloat();
      iT = 0.0;
    }
    else if (cmd.startsWith("SPP:")) {
      P_SET = cmd.substring(4).toFloat();
      iP = 0.0;
    }
    else if (cmd.startsWith("KPT:")) {
      KpT = cmd.substring(4).toFloat();
      iT = 0.0;
    }
    else if (cmd.startsWith("KIT:")) {
      KiT = cmd.substring(4).toFloat();
      iT = 0.0;
    }
    else if (cmd.startsWith("KPP:")) {
      KpP = cmd.substring(4).toFloat();
      iP = 0.0;
```

```
    }
    else if (cmd.startsWith("KIP:")) {
      KiP = cmd.substring(4).toFloat();
      iP = 0.0;
    }
  }
}

void setup() {
  Serial.begin(9600);
  Wire.begin();

  TCCR2B = TCCR2B & 0b11111000 | 0x01;

  pinMode(fan1DirPin, OUTPUT);
  pinMode(fan2DirPin, OUTPUT);
  pinMode(fan1BrakePin, OUTPUT);
  pinMode(fan2BrakePin, OUTPUT);
  pinMode(fan1PwmPin, OUTPUT);
  pinMode(fan2PwmPin, OUTPUT);

  digitalWrite(fan1DirPin, LOW);
  digitalWrite(fan2DirPin, LOW);
  digitalWrite(fan1BrakePin, LOW);
  digitalWrite(fan2BrakePin, LOW);

  analogWrite(fan1PwmPin, pwm1);
  analogWrite(fan2PwmPin, pwm2);

  pinMode(fan1TachPin, INPUT_PULLUP);
```

```
  pinMode(fan2TachPin, INPUT_PULLUP);

  enableInterrupt(fan1TachPin, fan1ISR, FALLING);
  enableInterrupt(fan2TachPin, fan2ISR, FALLING);

  Serial.println("time_s,temp_ref,temp_c,pressure_ref,pressure_pa,rpm_ref
1,rpm_ref2,rpm1,rpm2,pwm1,pwm2");
}

void loop() {
  readSerialCommands();

  unsigned long nowMs = millis();

  float tempRaw = readTemperatureLM35();
  float pressureRaw = readPressureSDP610();

  if (isnan(Tf)) Tf = tempRaw;
  Tf += alphaT * (tempRaw - Tf);

  if (pressureRaw != -9999.0) {
    if (isnan(Pf)) Pf = pressureRaw;
    Pf += alphaP * (pressureRaw - Pf);
  }

  // OUTER LOOP: temperature + pressure to RPM references
  if (nowMs - lastOuterMs >= outerLoopMs) {
    float dtOuter = (nowMs - lastOuterMs) / 1000.0;
    lastOuterMs = nowMs;

    // temperature error: if measured temp is above 25, increase fan speed
```

```
  float eT = deadbandError(Tf, T_SET, T_BAND);

  if (eT < 0) eT = 0;

  iT += KiT * eT * dtOuter;
  iT = constrain(iT, 0, 2000);

  float uT = KpT * eT + iT;

  // pressure control changes difference between fan speeds
  float eP = deadbandError(Pf, P_SET, P_BAND);

  iP += KiP * eP * dtOuter;
  iP = constrain(iP, -800, 800);

  float deltaRPM = -(KpP * eP + iP) * 20.0;

  rpmRefBase = rpmMin + uT;
  rpmRefBase = constrain(rpmRefBase, rpmMin, rpmMax);

  rpmRef1 = constrain(rpmRefBase + deltaRPM, rpmMin, rpmMax);
  rpmRef2 = constrain(rpmRefBase - deltaRPM, rpmMin, rpmMax);
}

// INNER LOOP: RPM references to PWM
if (nowMs - lastSpeedMs >= speedLoopMs) {
  unsigned long dtMs = nowMs - lastSpeedMs;
  lastSpeedMs = nowMs;

  noInterrupts();
```

```
unsigned long p1 = pulseCount1;
unsigned long p2 = pulseCount2;
pulseCount1 = 0;
pulseCount2 = 0;
interrupts();

float dt = dtMs / 1000.0;

float rpm1 = (p1 * 60.0) / (pulsesPerRev * dt);
float rpm2 = (p2 * 60.0) / (pulsesPerRev * dt);

float eSpeed1 = rpmRef1 - rpm1;
iSpeed1 += eSpeed1 * dt;
iSpeed1 = constrain(iSpeed1, -5000, 5000);

float uSpeed1 = KpSpeed * eSpeed1 + KiSpeed * iSpeed1;

pwm1 = pwm1 + uSpeed1;
pwm1 = constrain(pwm1, pwmMin, pwmMax);
analogWrite(fan1PwmPin, pwm1);

float eSpeed2 = rpmRef2 - rpm2;
iSpeed2 += eSpeed2 * dt;
iSpeed2 = constrain(iSpeed2, -5000, 5000);

float uSpeed2 = KpSpeed * eSpeed2 + KiSpeed * iSpeed2;

pwm2 = pwm2 + uSpeed2;
pwm2 = constrain(pwm2, pwmMin, pwmMax);
analogWrite(fan2PwmPin, pwm2);
```

```
    float time_s = nowMs / 1000.0;


    Serial.print(time_s, 3);
    Serial.print(",");
    Serial.print(T_SET, 2);
    Serial.print(",");
    Serial.print(Tf, 2);
    Serial.print(",");
    Serial.print(P_SET, 2);
    Serial.print(",");
    Serial.print(Pf, 2);
    Serial.print(",");
    Serial.print(rpmRef1, 1);
    Serial.print(",");
    Serial.print(rpmRef2, 1);
    Serial.print(",");
    Serial.print(rpm1, 1);
    Serial.print(",");
    Serial.print(rpm2, 1);
    Serial.print(",");
    Serial.print(pwm1);
    Serial.print(",");
    Serial.println(pwm2);
  }
}
```

Appendix B – MATLAB Implementation

```
clear;
clc;
close all;

port = "COM7";
baudRate = 9600;

arduinoObj = serialport(port, baudRate);
configureTerminator(arduinoObj, "LF");
arduinoObj.Timeout = 2;

flush(arduinoObj);
pause(3);

T_SET = 24.0;
P_SET = 0.0;

KpT = 35.0;
KiT = 8.0;
KpP = 3.0;
KiP = 1.2;

timeData = [];
tempData = [];
tempRefData = [];
pressureData = [];
pressureRefData = [];
rpmRef1Data = [];
```

```
rpmRef2Data = [];
rpm1Data = [];
rpm2Data = [];

fig = figure("Name", "Temperature, Pressure and Fan Speed Control", ...
    "NumberTitle", "off", ...
    "Position", [100 100 1200 800]);

% Controls
uicontrol(fig, "Style", "text", "String", "Temperature Setpoint [°C]",
...
    "Position", [20 750 160 25]);
tempEdit = uicontrol(fig, "Style", "edit", "String", num2str(T_SET), ...
    "Position", [190 750 70 25]);

uicontrol(fig, "Style", "text", "String", "Pressure Setpoint [Pa]", ...
    "Position", [280 750 150 25]);
pressureEdit = uicontrol(fig, "Style", "edit", "String", num2str(P_SET),
...
    "Position", [440 750 70 25]);

uicontrol(fig, "Style", "text", "String", "KpT", ...
    "Position", [530 750 40 25]);
kptEdit = uicontrol(fig, "Style", "edit", "String", num2str(KpT), ...
    "Position", [570 750 60 25]);

uicontrol(fig, "Style", "text", "String", "KiT", ...
    "Position", [650 750 40 25]);
kitEdit = uicontrol(fig, "Style", "edit", "String", num2str(KiT), ...
    "Position", [690 750 60 25]);
```

```
uicontrol(fig, "Style", "text", "String", "KpP", ...
    "Position", [770 750 40 25]);
kppEdit = uicontrol(fig, "Style", "edit", "String", num2str(KpP), ...
    "Position", [810 750 60 25]);

uicontrol(fig, "Style", "text", "String", "KiP", ...
    "Position", [890 750 40 25]);
kipEdit = uicontrol(fig, "Style", "edit", "String", num2str(KiP), ...
    "Position", [930 750 60 25]);

sendButton = uicontrol(fig, "Style", "pushbutton", ...
    "String", "Send Setpoints and Gains", ...
    "Position", [1010 745 170 35], ...
    "Callback", @sendValues);

% Plot 1: Temperature
ax1 = subplot(2,2,1);
hold(ax1, "on");
grid(ax1, "on");
title(ax1, "Temperature Control");
xlabel(ax1, "Time [s]");
ylabel(ax1, "Temperature [°C]");
tempRefLine = plot(ax1, NaN, NaN, "LineWidth", 1.5);
tempLine = plot(ax1, NaN, NaN, "LineWidth", 1.5);
legend(ax1, "Set temperature", "Measured temperature", "Location",
"best");

% Plot 2: Pressure
ax2 = subplot(2,2,2);
hold(ax2, "on");
grid(ax2, "on");
```

```
title(ax2, "Pressure Control");
xlabel(ax2, "Time [s]");
ylabel(ax2, "Pressure [Pa]");
pressureRefLine = plot(ax2, NaN, NaN, "LineWidth", 1.5);
pressureLine = plot(ax2, NaN, NaN, "LineWidth", 1.5);
legend(ax2, "Set pressure", "Measured pressure", "Location", "best");

% Plot 3: Fan 1 speed
ax3 = subplot(2,2,3);
hold(ax3, "on");
grid(ax3, "on");
title(ax3, "Fan 1 Speed Control");
xlabel(ax3, "Time [s]");
ylabel(ax3, "Speed [RPM]");
rpmRef1Line = plot(ax3, NaN, NaN, "LineWidth", 1.5);
rpm1Line = plot(ax3, NaN, NaN, "LineWidth", 1.5);
legend(ax3, "Fan 1 reference", "Fan 1 measured", "Location", "best");

% Plot 4: Fan 2 speed
ax4 = subplot(2,2,4);
hold(ax4, "on");
grid(ax4, "on");
title(ax4, "Fan 2 Speed Control");
xlabel(ax4, "Time [s]");
ylabel(ax4, "Speed [RPM]");
rpmRef2Line = plot(ax4, NaN, NaN, "LineWidth", 1.5);
rpm2Line = plot(ax4, NaN, NaN, "LineWidth", 1.5);
legend(ax4, "Fan 2 reference", "Fan 2 measured", "Location", "best");

disp("Reading data from Arduino...");
```

```
while ishandle(fig)
    try
        line = readline(arduinoObj);
    catch
        continue;
    end

    line = string(strtrim(line));

    if strlength(line) == 0 || contains(line, "time_s")
        continue;
    end

    data = split(line, ",");

    if numel(data) == 11
        t = str2double(data(1));
        tempRef = str2double(data(2));
        temp = str2double(data(3));
        pressureRef = str2double(data(4));
        pressure = str2double(data(5));
        rpmRef1 = str2double(data(6));
        rpmRef2 = str2double(data(7));
        rpm1 = str2double(data(8));
        rpm2 = str2double(data(9));

        if any(isnan([t tempRef temp pressureRef pressure rpmRef1 rpmRef2
rpm1 rpm2]))
            continue;
        end
```

```
        timeData(end+1) = t;
        tempRefData(end+1) = tempRef;
        tempData(end+1) = temp;
        pressureRefData(end+1) = pressureRef;
        pressureData(end+1) = pressure;
        rpmRef1Data(end+1) = rpmRef1;
        rpmRef2Data(end+1) = rpmRef2;
        rpm1Data(end+1) = rpm1;
        rpm2Data(end+1) = rpm2;

        set(tempRefLine, "XData", timeData, "YData", tempRefData);
        set(tempLine, "XData", timeData, "YData", tempData);

        set(pressureRefLine, "XData", timeData, "YData",
pressureRefData);
        set(pressureLine, "XData", timeData, "YData", pressureData);

        set(rpmRef1Line, "XData", timeData, "YData", rpmRef1Data);
        set(rpm1Line, "XData", timeData, "YData", rpm1Data);

        set(rpmRef2Line, "XData", timeData, "YData", rpmRef2Data);
        set(rpm2Line, "XData", timeData, "YData", rpm2Data);

        drawnow limitrate;
    end
end

function sendValues(~, ~)
    tempSet = get(tempEdit, "String");
    pressureSet = get(pressureEdit, "String");
```

```
    kpt = get(kptEdit, "String");
    kit = get(kitEdit, "String");
    kpp = get(kppEdit, "String");
    kip = get(kipEdit, "String");

    writeline(arduinoObj, "SPT:" + tempSet);
    writeline(arduinoObj, "SPP:" + pressureSet);
    writeline(arduinoObj, "KPT:" + kpt);
    writeline(arduinoObj, "KIT:" + kit);
    writeline(arduinoObj, "KPP:" + kpp);
    writeline(arduinoObj, "KIP:" + kip);

    disp("Sent new setpoints and gains to Arduino.");
end
```